

**Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Факультет Технологий искусственного интеллекта**

**Дисциплина:**

«Web-разработка: Frontend»

**ЛАБОРАТОРНАЯ РАБОТА № 3**

**Выполнил:**

Дубовик Никита Спартакович, студент группы J3210

---

(подпись)

# Отчёт по использованию моковой базы данных db.json

## 1. Введение

Файл db.json представляет собой фиктивную (mock) базу данных, используемую в связке с инструментом [JSON Server](#). JSON Server запускается локально на порту 3000 и предоставляет REST API для доступа к данным из db.json , имитируя полноценный бэкенд.

В проекте фронтенд обращается к этому API через слой сервисов @/services/api.js

## 2. Структура коллекций в db.json

Ранее в базе данных хранились только пользователи. Во время предыдущего ревью было сказано расширить базу данных, чтобы всё что нужно из frontend её использовало.

Файл содержит следующие коллекции:

| Коллекция    | Назначение                                                             |
|--------------|------------------------------------------------------------------------|
| users        | Учётные записи (студенты и преподаватели)                              |
| instructors  | Детальная информация о преподавателях (связь с users )                 |
| courses      | Курсы, включая описания, уровень, цену, рейтинг и ссылку на instructor |
| user_courses | Связка «пользователь-курс» (прогресс, статус)                          |
| discussions  | Обсуждения по курсам (темы, ответы, лайки)                             |
| materials    | Учебные материалы (файлы, ссылки)                                      |
| assignments  | Задания с дедлайнами и максимальными баллами                           |
| certificates | Выданные сертификаты                                                   |
| reviews      | Отзывы студентов на курсы                                              |

Основные связи:

- courses.instructorId → instructors.id
- instructors.userId → users.id
- user\_courses.userId → users.id , user\_courses.courseId → courses.id
- discussions.courseId → courses.id , discussions.userId → users.id
- materials.courseId → courses.id
- assignments.courseId → courses.id
- certificates.userId → users.id , certificates.courseId → courses.id
- reviews.courseId → courses.id , reviews.userId → users.id

## 3. Сервисный слой ( api.js )

Все запросы к `db.json` осуществляются через экземпляр `axios` с базовым URL `http://localhost:3000`. В файле `@/services/api.js` определены следующие функции, каждая из которых формирует запрос к конкретному эндпоинту JSON Server:

| Функция                                           | Метод HTTP | Эндпоинт JSON Server       |
|---------------------------------------------------|------------|----------------------------|
| <code>register(userData)</code>                   | POST       | <code>/signup</code>       |
| <code>login(email, password)</code>               | POST       | <code>/login</code>        |
| <code>logout()</code>                             | –          | – (клиентская очистка)     |
| <code>getCourses(params)</code>                   | GET        | <code>/courses</code>      |
| <code>getCourseById(id)</code>                    | GET        | <code>/courses/:id</code>  |
| <code>getPopularCourses(limit)</code>             | GET        | <code>/courses</code>      |
| <code>getMaterials(courseId)</code>               | GET        | <code>/materials</code>    |
| <code>getAssignments(courseId)</code>             | GET        | <code>/assignments</code>  |
| <code>getDiscussions(courseId)</code>             | GET        | <code>/discussions</code>  |
| <code>getUserByEmail(email)</code>                | GET        | <code>/users</code>        |
| <code>getInstructorByUserId(userId)</code>        | GET        | <code>/instructors</code>  |
| <code>getUserCourses(userId)</code>               | GET        | <code>/user_courses</code> |
| <code>getCertificates(userId)</code>              | GET        | <code>/certificates</code> |
| <code>getCoursesByInstructor(instructorId)</code> | GET        | <code>/courses</code>      |
| <code>getUserCoursesByCourseIds(courseIds)</code> | GET        | <code>/user_courses</code> |
| <code>getMaterialsByCourseIds(courseIds)</code>   | GET        | <code>/materials</code>    |
| <code>getReviewsByCourseIds(courseIds)</code>     | GET        | <code>/reviews</code>      |

**Особенности:**

- Аутентификация имитируется через эндпоинты `/login` и `/signup`
- Для коллекций, поддерживающих пагинацию (GET запросы), используется заголовок `x-total-count`, извлечённый функцией `extractTotalCount`.
- Отношения раскрываются через параметр `_expand=instructor` или `_expand=user`, что позволяет JSON Server автоматически подгружать связанные данные.

## 4. Использование `db.json` в компонентах приложения

### 4.1. Каталог курсов ( `CatalogView.vue` )

- **Функции:** `getCourses(params)`
- **Назначение:** загрузка и фильтрация списка курсов.
- **Параметры запроса:** строятся динамически на основе выбранных пользователем фильтров (предмет, уровень, длительность, цена, поисковый запрос, сортировка).

- **Результат:** отображение пагинированного списка курсов с информацией о преподавателе.

## 4.2. Детальная страница курса ( `CourseView.vue` )

- **Функции:**
  - `getCourseById(id)` – получение карточки курса.
  - `getMaterials(courseId)` – учебные материалы.
  - `getAssignments(courseId)` – задания.
  - `getDiscussions(courseId)` – обсуждения (авторы раскрываются через `_expand=user` ).
- **Назначение:** полная информация о курсе, включая вкладки «Обзор», «Материалы», «Задания», «Обсуждения».

## 4.3. Главная страница ( `HomeView.vue` )

- **Функция:** `getPopularCourses(3)` .
- **Назначение:** вывод трёх самых популярных курсов (по количеству студентов).

## 4.4. Профиль студента ( `StudentProfileView.vue` )

- **Функции:**
  - `getUserByEmail(email)` – получение данных пользователя из `/users` .
  - `getUserCourses(userId)` – прогресс по курсам из `/user_courses` .
  - Для каждого `courseId` вызывается `getCourseById(id)` для получения названий и деталей курсов.
  - `getCertificates(userId)` – сертификаты из `/certificates` , затем обогащаются данными курсов.
- **Назначение:** личный кабинет студента с вкладками «Мои курсы» (активные/завершённые) и «Сертификаты».

## 4.5. Профиль преподавателя ( `TeacherProfileView.vue` )

- **Функции:**
  - `getUserByEmail(email)` → `/users` .
  - `getInstructorById(userId)` → `/instructors` (поиск профиля преподавателя).
  - `getCoursesByInstructor(instructorId)` → `/courses` (курсы преподавателя).
  - `getUserCoursesByCourseIds(courseIds)` → `/user_courses` (прогресс студентов по этим курсам).
  - `getReviewsByCourseIds(courseIds)` → `/reviews` (отзывы на курсы).
  - `getMaterialsByCourseIds(courseIds)` → `/materials` (материалы курсов).
- **Назначение:** личный кабинет преподавателя с вкладками «Мои курсы», «Материалы», «Студенты».

## 4.6. Аутентификация

- **Модальные окна входа и регистрации** ( `LoginModal.vue` , `RegisterModal.vue` ) вызывают `login()` и `register()` из `useAuth()` .
  - Хранилище `auth.js` использует те же функции `apiLogin` и `apiRegister` из сервисного слоя.
- 

# Отчёт по повышению Accessibility сайта

## 1. Введение

Во время предыдущего ревью было подготовлено отчёт, где наглядно показать, что было улучшено и провести сравнение Lighthouse отчётов до и после.

## 2. Семантическая разметка и корректная структура заголовков

### Проблемы в старом HTML:

- Использование заголовков без логической иерархии (например, `<h5>` вместо `<h2>` для заголовков разделов).
- Отсутствие семантических элементов ( `<section>` , `<nav>` , `<main>` ) или их неправильное использование.
- Блоки контента не имели идентификаторов для связи с лейблами.

### Улучшения в Vue-приложении:

- Вся структура документа обернута в семантические теги: `<header role="banner">` , `<nav role="navigation">` , `<main role="main">` , `<footer role="contentinfo">` .
- Каждый раздел (hero, популярные курсы, отзывы) помечен как `<section>` с атрибутом `aria-labelledby` , указывающим на `id` заголовка.

```
<!-- HomeView.vue -->
<section class="hero ..." aria-labelledby="hero-title">
  ...
  <h1 id="hero-title" class="display-3 ...">Освойте <span>машинное обучение</span>
</h1>
</section>
```

- Установлена правильная иерархия заголовков: после единственного `<h1>` на странице следуют `<h2>` , `<h3>` в соответствии с вложенностью разделов (например, в `CourseView.vue` «О курсе» — `<h2>` , «Что вы изучите» — `<h3>` ).

## 2. Навигация и доступность

### Проблемы в старом HTML:

- Основное меню не имело атрибутов `role` и `aria-*` , что затрудняло ориентацию для скринридеров.
- Выпадающее меню профиля не информировало о своём состоянии.

## Улучшения:

- Навигационная панель получила `role="navigation"` и `aria-label="Основная навигация"`.
- Пункты меню сгруппированы в список с `role="list"` и `role="listitem"`.
- Выпадающее меню профиля ( `TheHeader.vue` ) теперь корректно управляется:

```
<a ... aria-expanded="true/false" aria-haspopup="true" aria-controls="accountMenu"
@click.prevent="toggleDropdown">
```

и само меню имеет `role="menu"`, `role="menuitem"` для пунктов. Активный пункт помечается `aria-current="page"`.

- Для кнопки переключения темы добавлены `aria-label="Переключить тему"` и `aria-pressed` (в `composable useTheme`), что позволяет программам экранного доступа объявлять состояние.
- Все интерактивные элементы (кнопки, ссылки) доступны с клавиатуры, а для элементов без видимого текста задан `aria-label` (например, `aria-label="Заккрыть окно входа"`, `aria-label="Показать пароль"`).

## 3. Формы и их доступные метки

### Проблемы в старом HTML:

- Поля ввода не имели связанных `<label>`, только `placeholder`.
- Элементы выбора ( `<select>` ) были без подписей.
- Группы чекбоксов фильтров не были объединены семантически.

## Улучшения:

- Все поля в формах регистрации, входа и фильтрации курсов получили явные `<label>` с корректным атрибутом `for`. При необходимости визуально скрытые метки добавляются через класс `visually-hidden`, оставаясь доступными для скринридеров.

```
<!-- CatalogView.vue -->
<label for="sortSelect" class="visually-hidden">Сортировка курсов</label>
<select id="sortSelect" ...>...</select>
```

- Группы фильтров по предмету, уровню и длительности снабжены `role="group"` и `aria-labelledby`, ссылающимся на заголовок группы.

```
<div class="fw-bold mb-2" id="subjectLabel">Предмет</div>
<div role="group" aria-labelledby="subjectLabel">...</div>
```

- Блок цен обернут в `<fieldset>` с `<legend>`, что объединяет поля минимальной и максимальной цены и чекбокс «Только бесплатные».
- Для поля ввода пароля при регистрации добавлен прогресс-бар с ролью `progressbar`, атрибутами `aria-valuenow`, `aria-valuemin`, `aria-valuemax` и визуальным текстом, описывающим надёжность пароля.
- Сообщения об ошибках валидации помечены классом `invalid-feedback` и связаны с соответствующим полем через атрибут `aria-describedby`.

## 4. Альтернативный текст и скрытие декоративных элементов

### Проблемы в старом HTML:

- Все иконки Font Awesome были вставлены как `<i class="fa-solid ..."></i>`, не имея `aria-hidden`, что заставляло скринридеры зачитывать бессмысленные символы.
- Изображения преподавателей и аватары не имели атрибута `alt`.

### Улучшения:

- Иконки полностью переведены на SVG-спрайт (`AppSvgSprite.vue`). Во всех местах использования добавлен `aria-hidden="true"`, сообщая вспомогательным технологиям, что иконка является декоративной.
- Для каждого информативного изображения (аватар преподавателя, скриншот курса) прописан осмысленный `alt`, формируемый динамически из данных (например, `:alt="instructorName"` в `CourseCard.vue`).
- Вместо иконок-ссылок (в footer для социальных сетей) используются ссылки с `aria-label`, а сами SVG скрыты от скринридеров.

```
<a href="#" aria-label="Наш канал в Telegram">
  <svg aria-hidden="true"><use href="#icon-telegram"></use></svg>
</a>
```

## 5. Управление динамическим контентом и живыми областями

### Проблемы в старом HTML:

- Результаты поиска и фильтрации менялись без уведомления скринридеров.

### Улучшения:

- Блок с количеством найденных курсов (`CatalogView.vue`) использует `aria-live="polite"` и `aria-atomic="true"`, благодаря чему любое изменение числа курсов сразу объявляется.
- Для уведомлений используется контейнер с `role="alert"`, что автоматически зачитывает сообщение при появлении.
- При загрузке данных показывается спиннер с текстом «Загрузка...», скрытым для визуальных пользователей (`visually-hidden`), но доступным для скринридеров через `role="status"`.

## 6. Доступность модальных окон

### Проблемы в старом HTML:

- В модальных окнах входа и регистрации использовались заголовки `<h5>`, что нарушало иерархию; кнопки закрытия не имели текстового описания.

### Улучшения:

- Заголовки модальных окон ( `LoginModal` , `RegisterModal` ) повышены до `<h3>` , что сохраняет структуру документа.
- Кнопка закрытия снабжена `aria-label="Закрыть окно входа"` и `aria-label="Закрыть окно регистрации"` .
- При открытии модального окна фокус перемещается на него, и с помощью `Bootstrap Modal` обеспечивается захват фокуса внутри диалога.

## 7. Списки и коллекции элементов

### Проблемы в старом HTML:

- Сетки карточек курсов и отзывов были просто `<div>` без семантического обозначения списка.

### Улучшения:

- Контейнеры карточек получили `role="list"` , а каждая карточка — `role="listitem"` . Это позволяет скринридерам объявлять количество элементов и облегчает навигацию:

```
<div class="row g-4" role="list">
  <CourseCard ... role="listitem" />
</div>
```

- Звёздочки рейтинга в отзывах теперь обернуты в `<div>` с `role="img"` и `aria-label="Рейтинг: 5 из 5 звезд"` , что даёт текстовое представление визуальной оценки.

## 8. Цветовая контрастность и поддержка тем

- Реализован переключатель тем (светлая/тёмная) с сохранением состояния, что помогает пользователям с нарушениями зрения подбирать комфортный контраст.
- Все интерактивные элементы имеют достаточный цветовой контраст, соответствующий стандартам.

## Заключение

В результате количество нарушений доступности, выявленных `Lighthouse` , сократилось до 0. Все ключевые улучшения, перечисленные в данном отчёте, внедрены непосредственно в компоненты нового SPA и подтверждены сравнением аудитов `Lighthouse` .

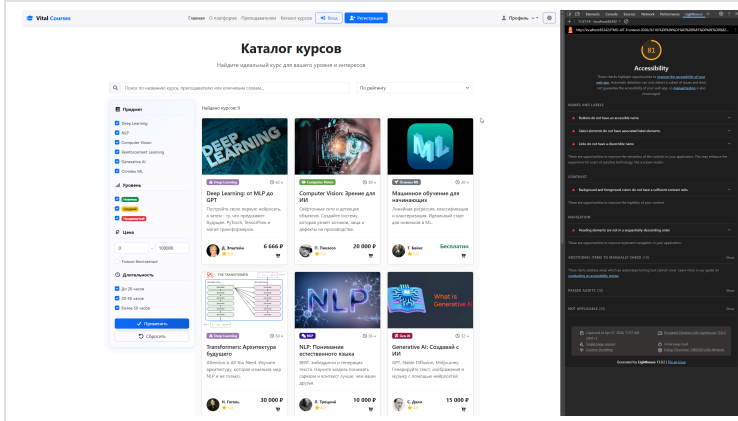
---

## Сравнение отчётов Lighthouse

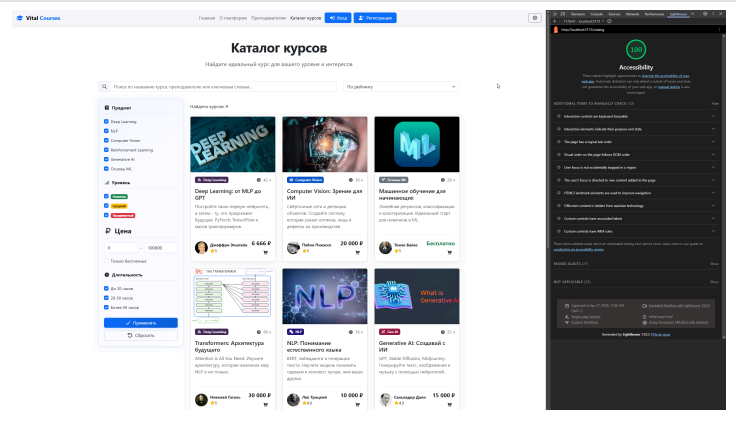
### Каталог



## Старая версия

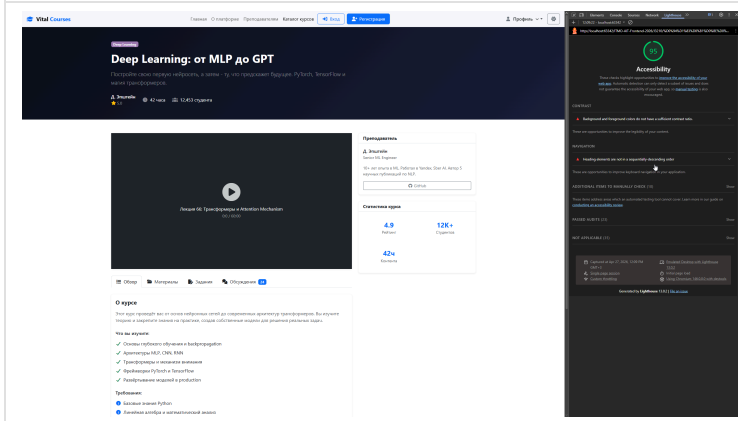


## Новая версия

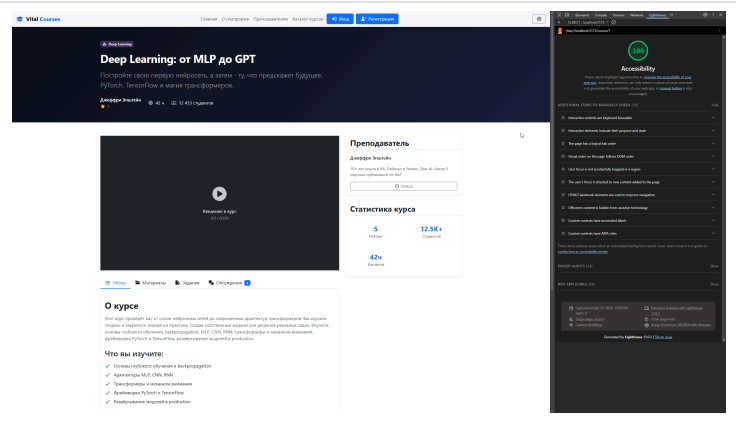


## Курс

## Старая версия

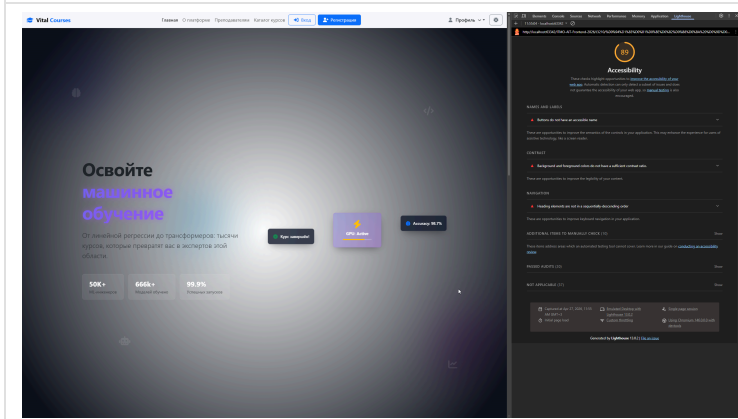


## Новая версия

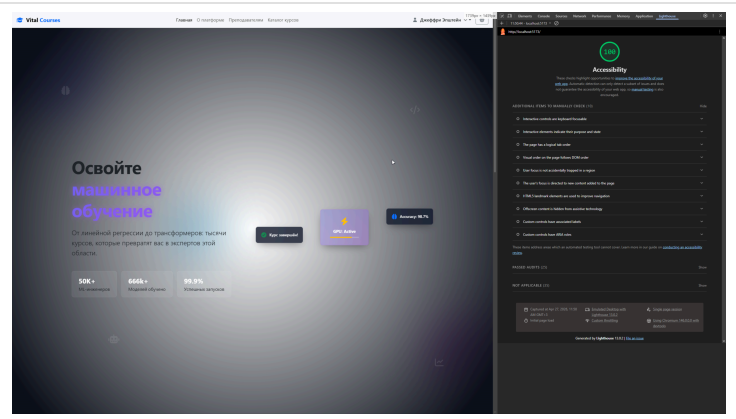


## Главная страница (Index)

## Старая версия

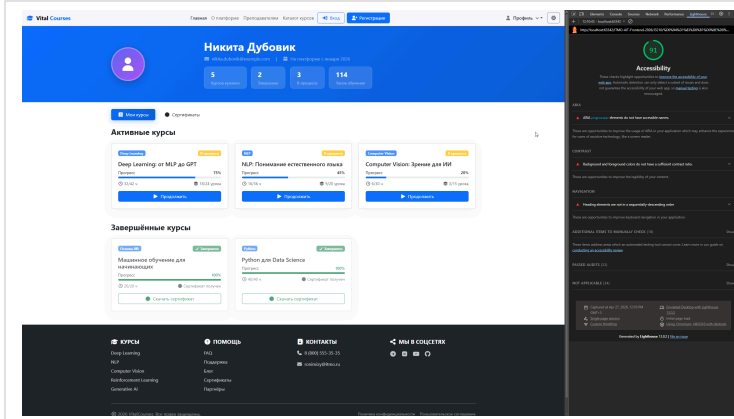


## Новая версия

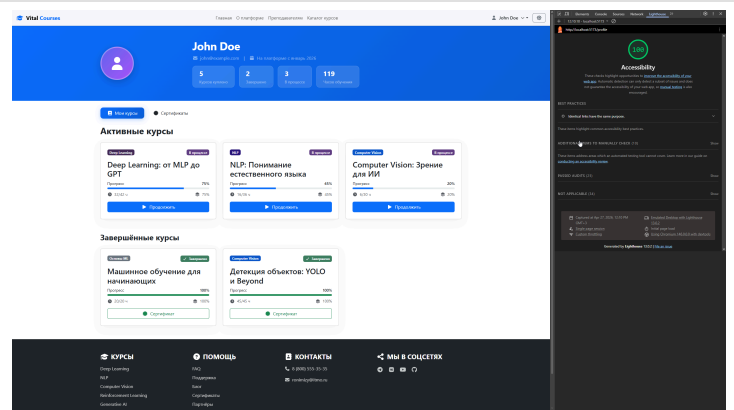


## Профиль пользователя

## Старая версия

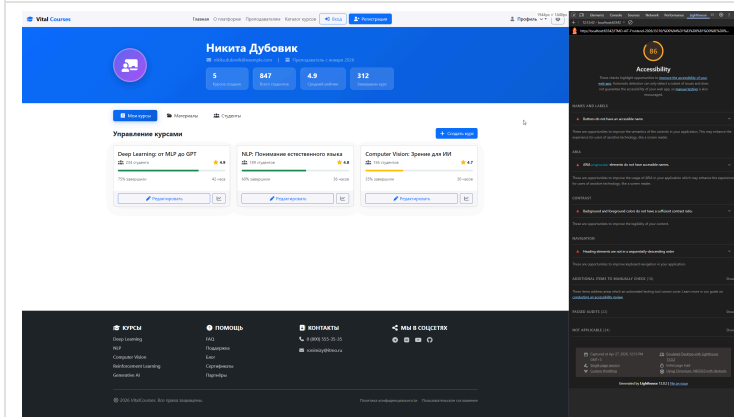


## Новая версия

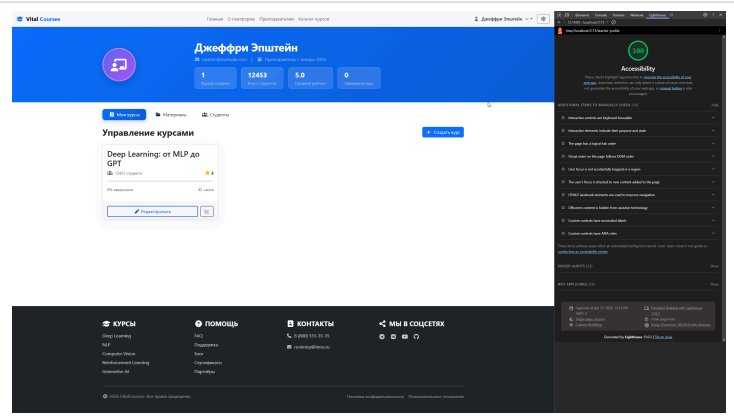


## Профиль преподавателя

## Старая версия



## Новая версия



# Отчёт по миграции сайта на Vue.js для создания SPA

## 1. Введение

Проект представляет собой одностороннее приложение образовательной платформы, построенное с использованием **Vue 3** (Composition API), **Vue Router**, **Pinia**, **Bootstrap 5** и взаимодействующее с REST API, эмулируемым `json-server-auth`.

## 2. Архитектура и организация проекта

### Структура каталогов

```
src/
├── assets/styles/           # Глобальные стили (main.css, index.css, catalog.css и т.д.)
├── components/
│   ├── common/            # Переиспользуемые UI-компоненты (AppSpinner, AppSvgSprite,
│   │                       TheHeader, TheFooter, LoginModal, RegisterModal, ToastContainer)
│   └── ...
```

```

├── courses/           # CourseCard, CourseFilter
│   └── profile/      # CourseProgressCard, CertificateCard
├── composables/      # useAuth, useTheme, useToast
├── router/           # Конфигурация маршрутов
├── services/         # API-клиент (api.js)
├── stores/           # Pinia-хранилище auth.js
├── utils/            # Вспомогательные функции (helpers.js)
├── views/            # Компоненты-страницы (HomeView, CatalogView, CourseView,
StudentProfileView, TeacherProfileView)
├── App.vue
└── main.js

```

Приложение инициализируется в `main.js`, где создаётся экземпляр Vue, подключаются Pinia, роутер и глобальные стили Bootstrap и собственные.

## 3. Ключевые возможности Vue.js, применённые в проекте

### 3.1 Компонентный подход

Проект разбит на множество мелких компонентов с чёткой ответственностью:

- **AppSvgSprite** - централизованный SVG-спрайт для иконок.
- **TheHeader / TheFooter** - навигация и футер.
- **LoginModal / RegisterModal** - модальные окна с собственной логикой валидации и взаимодействием с хранилищем аутентификации.
- **CourseCard, CourseProgressCard, CertificateCard** - отображение карточек курсов и прогресса.
- **ToastContainer** - рендеринг уведомлений через Bootstrap Toast.

### 3.2 Управление состоянием (Pinia)

Хранилище `auth` (`stores/auth.js`) управляет:

- состоянием пользователя и токена;
- геттерами `isAuthenticated`, `isTeacher`, `fullName`;
- действиями `login`, `register`, `logout`, `refreshUser`;
- персистентностью через `pinia-plugin-persistedstate`.

Данные хранилища используются в composable `useAuth()`, который предоставляет ссылки и методы, что позволяет компонентам реактивно реагировать на изменение статуса аутентификации.

### 3.3 Маршрутизация (Vue Router)

Определены маршруты:

- `/` - HomeView
- `/catalog` - CatalogView
- `/course/:id` - CourseView

- `/profile` - `StudentProfileView` (beforeEnter проверка аутентификации)
- `/teacher-profile` - `TeacherProfileView` (так же требует роль `teacher`)
- `/:pathMatch(.*)*` - редирект на главную

Использован `createWebHistory`, `scrollBehavior` для прокрутки в начало страницы при переходе. Навигация реализована через `<router-link>` и программный переход в `beforeEnter`.

## 3.4 Composables

- `useAuth()` - обёртка над хранилищем аутентификации.
- `useTheme()` - управление темой (светлая/тёмная) через атрибут `data-bs-theme` на `<html>`, сохранение в `localStorage`. Возвращает `isDark`, `toggleTheme`.
- `useToast()` - система уведомлений: глобальный массив `toasts`, функции `showToast`, `removeToast`. Автоматическое удаление через 4 секунды.

## 3.5 Обработка форм и валидация

В `LoginModal` и `RegisterModal` реализована валидация на стороне клиента с отображением ошибок через `invalid-feedback` и условные классы. Индикатор загрузки (`spinner`) при отправке. В `RegisterModal` продвинутая оценка надёжности пароля с прогресс-баром и текстовым описанием.

## 3.6 Иконки

Все иконки вынесены в единый спрайт `AppSvgSprite.vue` и используются через `<svg><use href="#icon-..."></use></svg>`. Такой подход уменьшает размер разметки и упрощает поддержку.

# 4. Основные страницы и функциональность

## 4.1 Главная страница (HomeView)

- Hero-секция с анимированными элементами и статистикой.
- Популярные курсы (загружаются через `getPopularCourses` с сортировкой по количеству студентов).
- Блок «Как стать ML-гуру» и отзывы (сделал статичными).

## 4.2 Каталог курсов (CatalogView)

- Полноценная фильтрация и поиск.
- Пагинация.
- Отображение карточек курсов с информацией о преподавателе, рейтинге, цене.
- Адаптивная сетка (Bootstrap grid).

## 4.3 Детальная страница курса (CourseView)

- Загрузка данных курса, материалов, заданий, обсуждений.
- Табы с разделами: обзор, материалы, задания, обсуждения.

- Отображение информации о преподавателе и статистике курса.
- Состояния загрузки ( `AppSpinner` ), ошибки, отсутствия данных.

## 4.4 Профиль студента (`StudentProfileView`)

- Интеграция с `Pinia`: получение текущего пользователя, затем детальной информации через `getUserByEmail` .
- Загрузка записей пользователя на курсы, сертификатов, и самих курсов.
- Вычисляемые свойства: количество курсов, часов, активных/завершённых.
- Компоненты `CourseProgressCard` и `CertificateCard` .

## 4.5 Профиль преподавателя (`TeacherProfileView`)

- Аналогичная шапка с собственной статистикой.
- Табы: «Мои курсы» (карточки с прогрессом студентов), «Материалы» (таблица с возможностью загрузки/удаления), «Студенты» (статистика и отзывы).
- Вычисление процента завершения курса.
- Загрузка материалов и отзывов параллельно через `batch`-запросы.